DATA PRODUCT CASE STUDY · STELLARALGO

PRIME: one governed data layer for a multi-tenant product

From week-old, disputed numbers to one layer every team, dashboard and AI reads from.

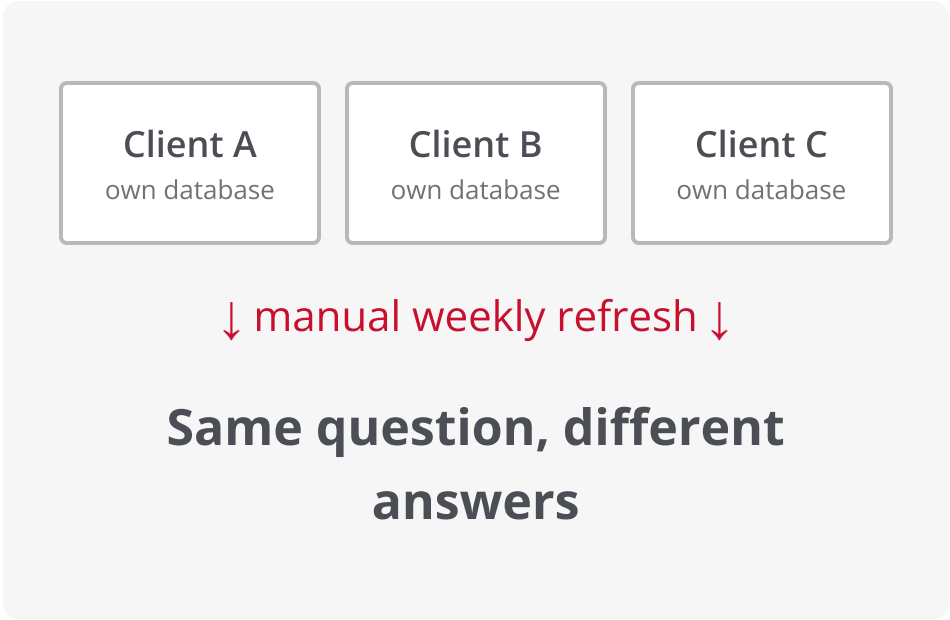
Cody Chandler
Data and Business Analyst interview · The City of Calgary

I'll answer them in order

1. What business problem was I solving?
2. What is the data product?
3. How did I approach it?
4. What challenges came up?
5. What am I most proud of?
6. What would I improve with more time?

Then five minutes live: the same approach, rebuilt on City of Calgary open data.

What business problem was I solving?



Our multi-tenant product kept each client's data in its own database, so accurate internal reporting was hard. Reports were up to **7 days old**, a cross-client question meant visiting every system, and nobody owned the numbers.

a **Why it mattered**

Teams and clients worked from different numbers, and every cross-client question needed engineering.

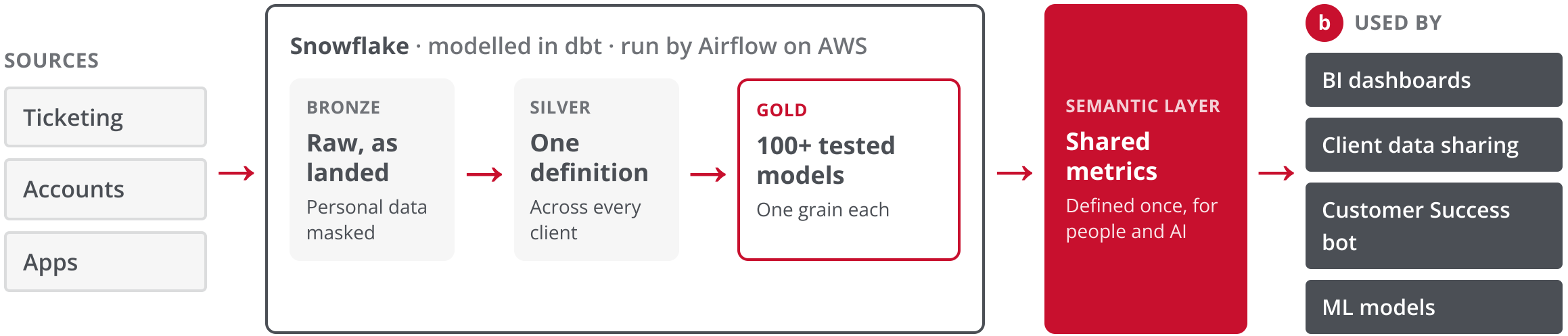
b **Users and interested parties**

Customer Success · Product · Clients · Analysts · Data Engineering

The need: one consolidated analytical source of truth, so internal reporting is consistent and reliable at scale.

DATA THAT IS **Trusted, Fresh, Owned, Safe, Self-serve**

What is the data product?



a What it does

Consolidates every client's data into one governed, tested layer, refreshed daily, that any approved team or tool can use.

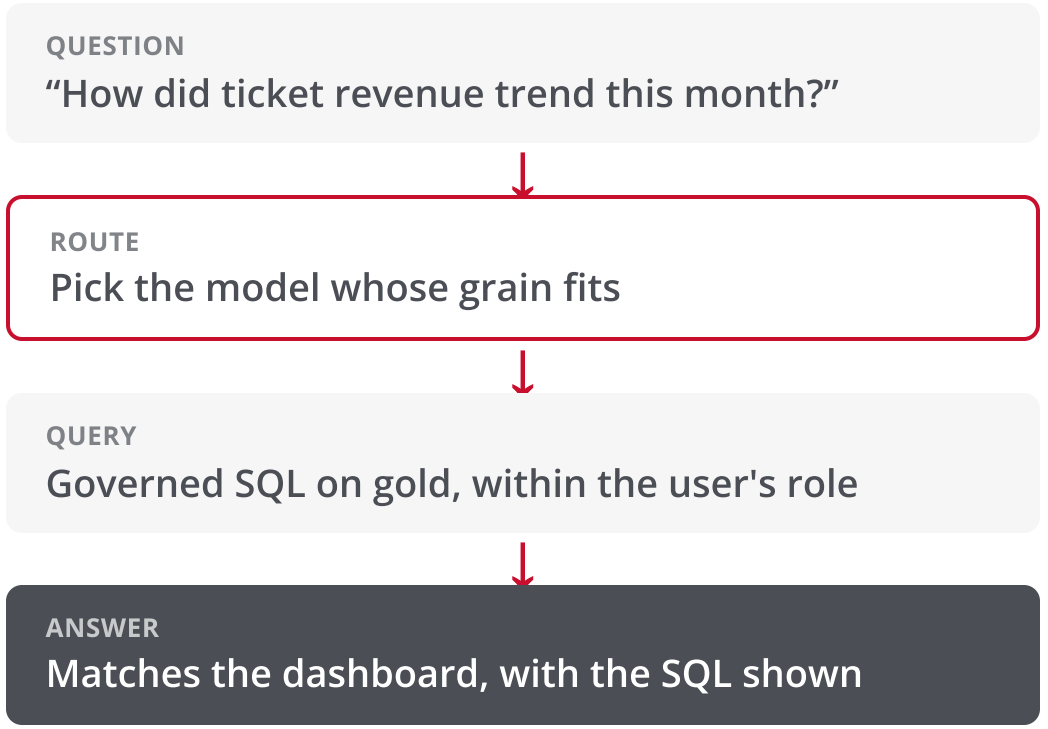
b How people use it: ask in plain English

SEMANTIC LAYER: ONE GRAIN PER MODEL

Property by day
One row per property, per day
e.g. ticket revenue, attendance

Campaign
One row per campaign
e.g. sends, conversions

More models
Each with one documented grain
Same definitions as the dashboards



Shipped first as an internal Customer Success bot. I led discovery for Stellar Insights, the client-facing version.

C What value it delivered

✓ FRESH

~~7 days~~

< 12 hrs

Data freshness

✓ TRUSTED

100+

Tested models, replacing a database per client

✓ OWNED

30 min

To flag a missed refresh, before a client notices

✓ SELF-SERVE

No ticket

For routine Customer Success questions

✓ **SAFE** Personal data masked on landing; every person and tool sees only what their role allows.

Customer Success, Product and clients now work from the same numbers, on a schedule they can rely on.

How did I approach it?

Requirements
What each team needed to trust a number.

→

Strategy
Fix the data once, not every report. Contract before charts.

→

Plan
Bronze, silver, gold. One grain per gold table.

→

Build
Airflow and dbt replaced the manual weekly refresh.

→

Review and iterate
Monitoring finds issues; fix them in the layer, once.

a Sourcing and preparing the data

Three source systems, landed raw with personal data masked, then mapped client by client into one dimensional model and tested.

b Methods and tools

Snowflake · dbt · Airflow · AWS · Medallion layers · Semantic layer · Data contract · LLMs

What challenges came up, and how did I handle them?

Type	Challenge	How I solved it
Technical	Every client system had its own shape.	One dimensional model all clients map into.
Analytical	The same metric meant different things to different teams.	One definition in silver, one documented grain per gold table.
People	Nobody owned freshness or accuracy.	A data contract I wrote: owners, targets per domain, three incident tiers.

What am I most proud of?

Governance stopped being overhead and became the reason people could self-serve.

One tested layer feeds dashboards, client data and the AI, with no separate pipeline for each. That's why the AI's answers match the dashboards.

What would I improve with more time?

Measure from day one

Baseline escalations and Customer Success hours before launch, so the value is a number.

Insights for clients

Take the internal bot to clients as Stellar Insights, on the same layer.

More semantic models

Cover more domains, each with one grain, so the AI can answer more.

The same work at the City: find where numbers disagree, agree one definition with the owners, and make it self-serve.

The same approach, live on City of Calgary data

1 Approach: the pipeline and its lineage

2 Dashboard: a 43% jump that wasn't real

3 Predict: a live XGBoost model

4 Ask the data: plain English, with its working

Airflow · dbt · DuckDB · 27 tests · XGBoost · AI agent · codychandler.ca/data-demos/calgary-transit

Thank you. Questions?

Case study	**codychandler.ca/work/stellaralgo-prime**
Live demo	**codychandler.ca/data-demos/calgary-transit**